

Why Google Colab?

We chose Google Colab for this project because it provides high-performance computing resources. Training a convolutional neural network on a large dataset requires a GPU. Since personal computers often lack the necessary hardware power, Colab allowed us to train our models faster. It also provides a ready-to-use environment for Python and deep learning libraries like PyTorch.

Data and Code Integration

The NIH ChestX-ray14 dataset is very large, so we developed a specific strategy to manage it:

- **Google Drive Connection:** We uploaded the dataset to Google Drive. We used a mount command to connect the Drive to the Colab environment. This way, the code could access the images without uploading them every session.
- **Modular Code Loading:** We stored our CNN architecture and custom scripts in a folder on Drive. We added this folder to the system path in Colab to import our models easily.
- **Data Extraction:** Because reading thousands of small files from Drive can be slow, we extracted the zip files directly into the Colab disk at the start of each session.

Technical Challenges

Using a cloud-based environment brought some difficulties during the development process:

- **GPU Access Limits:** Google Colab has daily limits for GPU usage. Sometimes we could not continue training because the GPU hours were finished.
- **VRAM and Memory Issues:** High-resolution images and deep models use a lot of video memory. We received several "Out of Memory" errors. We solved this by reducing the batch size.
- **Connection Timeouts:** Sessions disconnect after some time. We saved model checkpoints frequently to prevent losing our training progress when the session ended.
- **Processing Delays:** Moving data between the CPU and GPU created bottlenecks. We used optimized data loaders to make this process more efficient.

Overview of Jupyter Notebook Scripts

The Jupyter Notebook files in this project contain the complete pipeline for our experiments. Each script is organized into blocks that manage environment setup, data preprocessing, model training, and performance evaluation. These notebooks document the progress from a basic baseline model to an optimized system. They include training logs, loss curves, and final test results, such as F1 scores and AUC values for each disease class. By providing both the code and the output in a single file, these scripts allow for a clear and interactive review of the model development process.